Modeling Human Activity States Using Hidden Markov Models

Formative 2, Machine Learning · individual assignment (one student, one iPhone 13)

Ivan Manzi Ya Musana

GitHub: https://github.com/Manzi453/Formative1_TechinuqueII

1. Background and Motivation

Human activity recognition (HAR) turns the motion sensors that a phone already carries into a running description of what its owner is doing, without a camera. That matters for activity and health monitoring: measuring how much a person moves, noticing long stretches of inactivity and flagging unusual movement. My use case is a phone-only recognizer for four everyday states, **standing, walking, jumping and still**, trained and tested on my own iPhone 13 recordings. Activities are not independent snapshots: a person who is walking is very likely to be walking one second later, and some switches are more plausible than others, for example still to standing rather than still to jumping. A Hidden Markov Model (HMM) captures exactly this structure, with hidden states (the activity) that evolve through a transition matrix A and noisy sensor features (the observations) that depend on the state through emissions B. I implemented the HMM from scratch in numpy (forward-backward, Baum-Welch training and Viterbi decoding) and tested it on recordings it had never seen, where it decoded 91.5 % (65 of 71) of the windows correctly (Section 4).

2. Data Collection and Pre-processing

Videos (one per activity): Standing · Walking · Jumping · Still

I recorded 52 clips, 13 per activity, 6.7 to 17.0 s each (118.3 s standing, 130.1 s walking, 121.7 s jumping, 115.2 s still). The script `scripts/make_clean_dataset.py` builds the cleaned set and never modifies `Data/`. Every recording goes through the same rules: (1) trim both ends by the largest of 1.0, 0.75 and 0.5 s for which every activity reaches 90 s. No candidate does (1.0 s leaves standing at 76.9 s and still at 81.2 s; 0.75 s leaves standing at 86.1 s; 0.5 s leaves still at 88.2 s), so I used 0.5 s; (2) an edge-transient rule: drop an outermost 1 s segment (at most three per end) while its accelerometer-magnitude std exceeds twice the median of the clip, which removed 10 s in total from 6 clips, including the pick-up movement at the start of the worst recording (46.7 times the median); (3) centre-crop to at most 10 s and keep clips of at least 5 s; (4) rename to `<activity>_<NN>.csv` with columns `timestamp, ax, ay, az, gx, gy, gz`. Two recordings are excluded (`Data_clean/excluded.csv`): `standing_02-2026-10-28_17-23-27` is **mislabeled** (accelerometer-magnitude std 0.862 against a standing median of 0.089, so its signal looks like walking), and `still01-2026-10-28_17-23-58` is 4.76 s long after the edge rule. Table 1 summarizes the result.

Activity	Train clips	Test clips	Seconds (train + test)	Windows (train / test)
standing	10	2	92.3	58 / 17
walking	10	3	108.6	68 / 24
jumping	10	3	107.8	73 / 17
still	10	2	**88.2**	58 / 13
Total	**40**	**10**	**397.0**	**257 / 71**

*Table 1. Cleaned dataset (50 clips). Standing, walking and jumping reach the 90 s minimum; **still is 1.8 s short** (88.2 s). One more still recording of at least 11 s (10 s after trimming) would close the gap.*

Unseen test set. The 10 test clips are the hand-named recordings made in separate sessions (filename times 16:39 to 16:41 and 17:22 to 17:29 UTC; the metadata gives the timezone Africa/Kigali, 2 h ahead). They are never used for the normalization statistics, the model fit, Baum-Welch or the choice of variance floor and pi, and a programmatic check confirms that no test clip name or source recording appears in the training set. The edge-transient rule was added after an earlier run showed that the test errors sat at the start of clips; it uses only the signal and applies to every recording, but it means the test set did not stay completely out of my design decisions. Two training clips, `still_05` and `still_09`, have a standard-deviation profile closer to standing; the rules above are my only filters, so I kept them.

All data are mine: one person, one phone (iPhone 13, one device id in all 52 recordings). The files follow the Sensor Logger export format (appVersion 1.68, iOS 18.2.1) but the app name itself is not stored in the files. Accelerometer and gyroscope were logged at `sampleRateMs = 10`, i.e. 100 Hz. I **measured** the real rate of every cleaned clip from its timestamps: 100.118 Hz on average (range 100.105 to 100.124 Hz, std 0.005 Hz), with 9.98 to 9.99 ms between every pair of consecutive samples. One device at one uniform rate needs no harmonization or resampling (it would only add interpolation error), and I verified this in code instead of assuming it. The two streams were aligned by nearest timestamp (5 ms tolerance), which also resolved the one recording with 916 accelerometer but 917 gyroscope rows. I use the calibrated `Accelerometer.csv` and `Gyroscope.csv` only, because the uncalibrated

accelerometer file is on a different scale. Units are not stated in the files; I label them m/s² and rad/s, inferred from the signal scale. Table 2 summarizes the rate.

Phone	Nominal rate	Measured rate (mean, range)	Rate used
iPhone 13	100 Hz	100.118 Hz (100.105 to 100.124 Hz)	100.118 Hz, no resampling

Table 2. Sampling rate of the phone and the rate used for the window length and the FFT.

Each clip is cut into **2 s windows (200 samples at 100.118 Hz) with 50 % overlap** (hop 1 s), giving 257 training and 71 test windows. The length in samples is computed from the measured rate (2 s x 100.118 Hz, rounded). The 2 s length holds about three steps at my measured walking cadence (about 1.5 Hz), gives a 0.5 Hz FFT resolution (enough to separate walking from jumping at about 2 Hz) and is short enough that a window rarely spans two activities. Overlapping windows are correlated, so the train/test split is done by clip, never by window. Each window takes the label of its clip. Figure 1 shows a typical clip of each activity.

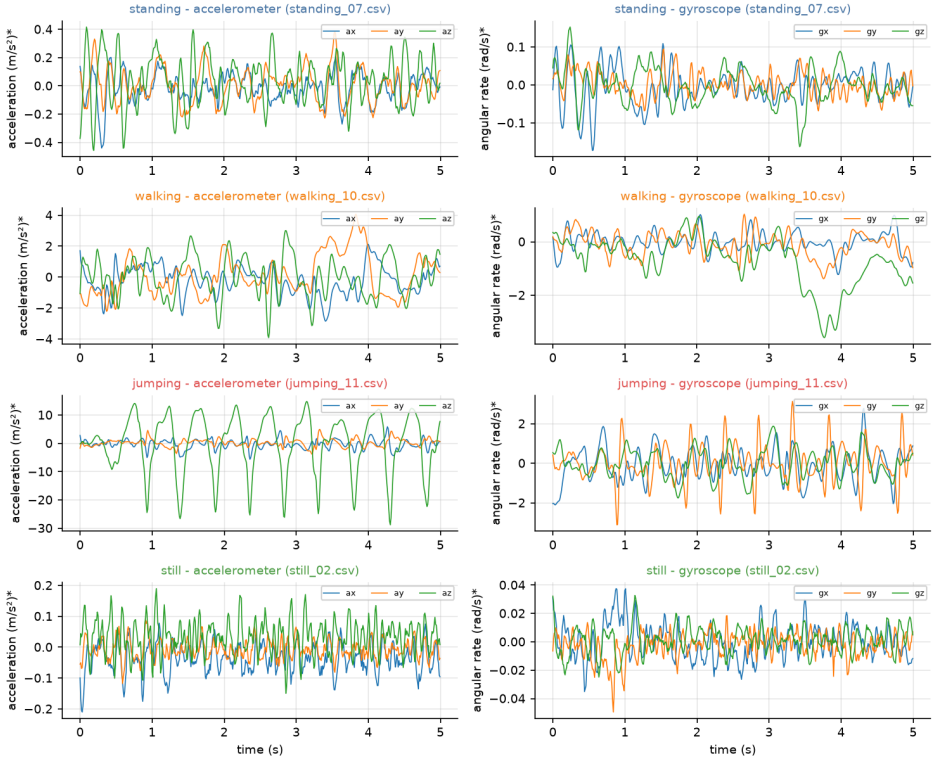


*Figure 1. Five seconds of accelerometer (left) and gyroscope (right) for a typical training clip of each activity. Each panel has its own y-scale; *units inferred, not documented in the files. Jumping shows a clear rhythm of about 2 Hz, mainly on the z axis, walking a weaker rhythm of about 1.5 Hz, and still and standing are mostly noise.*

Each video shows me performing the activity with the phone in my hand: standing upright and looking at the phone, walking around a room, repeated jumps on the spot, and sitting motionless on a stool. The descriptions are what the videos show; the sensor files carry no posture annotation (`Annotation.csv` is empty).

3. HMM Setup and Implementation

For each window and each sensor I compute 22 features, **44 in total** (Table 3). I take the dominant frequency per axis, not from the magnitude, because rectifying the magnitude folds the spectrum and was unstable for walking.

Feature(s), per sensor	Type	Why it helps
mean x, y, z	Time	Residual bias and posture drift; close to 0 for the gravity-free accelerometer
std x, y, z; RMS x, y, z	Time	Movement intensity: still < standing < walking < jumping
SMA (signal magnitude area)	Time	Whole-body activity intensity in one number
correlation xy, xz, yz	Time	Direction of motion: jumping is dominated by one axis, walking couples several axes
dominant frequency per axis (0.5 to 10 Hz)	FFT	Cadence: about 1.5 Hz for walking, about 2 Hz for jumping, noise-like for still and standing
spectral energy per axis	FFT	How much power sits in the motion band
spectral entropy per axis	FFT	Periodic movement gives a low entropy, sensor noise a high one

Table 3. The 22 features computed per sensor (accelerometer and gyroscope) for each 2 s window, 44 in total.

Normalization. I first take log10 of the 20 strictly positive amplitude features (std, RMS, SMA, energy), which differ by orders of magnitude between still and jumping, and then apply a **Z-score using the mean and standard deviation of the training windows only**; the test windows reuse those statistics. Z-scoring puts features with different units (Hz, m/s², rad/s, unit-free correlations) on one scale, which matters because the Gaussian emissions share one variance floor. A Gaussian with its own variance per state is invariant to rescaling, so the Z-score acts mainly through the floor and the numerics; the log transform is what reshapes the distributions.

The hidden states are Z = {standing, walking, jumping, still}; each observation X is a 44-dimensional Z-scored window vector. The emissions B are **diagonal Gaussians** with a variance floor, A is the 4x4 transition matrix and pi the initial distribution. Everything is plain numpy: log-space forward and backward passes, **Viterbi** in log-space, and **Baum-Welch**, which re-estimates A, the means and the variances and **stops when |ΔLL| < 1e-4** (a 500-iteration cap is only a safeguard). Correctness is tested in the notebook (section 7.2): the forward likelihood equals brute-force enumeration over all paths, Viterbi returns the brute-force best path and the textbook Healthy/Fever answer (Healthy, Healthy, Fever, probability 0.01512), and Baum-Welch never decreases the likelihood; the notebook also asserts that the rows of A and pi sum to 1 and that all parameters are finite. Baum-Welch trains the model and Viterbi decodes a window sequence into activity labels, which is exactly what the sensitivity, specificity and confusion-matrix evaluation scores.

The parameters are initialized from the labeled training windows (class means, variances, empirical transitions) and then refined by Baum-Welch. An activity change never occurs inside one of my clips, so the training sequences are **10 random shuffles of all 40 training clips concatenated** (seed 42); the diagonal of A therefore reflects clip length and the off-diagonal entries are artifacts of the shuffling. Two choices were made on the training clips only, with rules fixed in advance (leave-one-clip-out cross-validation with the normalizer re-fit in every fold). **Variance floor:** the largest floor with the best accuracy; the accuracy was 97.3 % for 0.01 to 0.1, 98.1 % for 0.25 and 98.4 % for 0.5, so the floor is 0.5 (in Z-score units). **Pi:** Baum-Welch was allowed to update pi only if that was strictly more accurate; it was not (98.4 % both ways), so pi stays fixed at the share of training clips per activity (0.25 each).

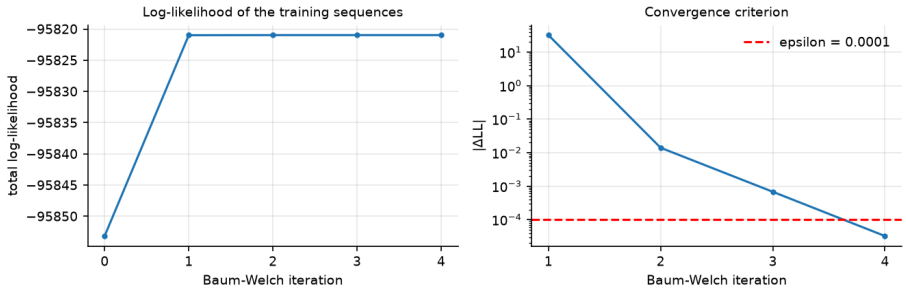


Figure 2. Baum-Welch on the training sequences: total log-likelihood per iteration (left) and the change |ΔLL| against the stopping threshold epsilon = 1e-4 (right).

Baum-Welch **converged after 4 iterations** (|ΔLL| = 3.2e-5 < 1e-4), the likelihood never decreased, and the total training log-likelihood rose from -95853.16 to -95820.97 (Figure 2). I checked that every state still decodes to its own activity. The test predictions are identical with and without Baum-Welch (65 of 71 correct in both cases), so **Baum-Welch changed the decisions on this dataset very little**. 79.5 % of the state variances sit exactly at the floor (the median un-floored within-class variance is only 0.062), so the model behaves much like a nearest-mean classifier.

4. Results and Interpretation

Each test clip is decoded separately with Viterbi and the window decisions are pooled (71 windows from 10 clips). Table 4 gives the per-activity results computed one-versus-rest from the confusion matrix in Figure 3.

Activity	Number of samples (windows)	Sensitivity	Specificity	Accuracy (one-vs-rest)
standing	17	1.000	0.926	0.944
walking	24	0.917	1.000	0.972
jumping	17	1.000	0.963	0.972
still	13	0.692	1.000	0.944
Overall accuracy	**71**			**0.915 (65 / 71)**

Table 4. Results on the unseen test clips. The 95 % Wilson interval for the overall accuracy is 82.8 % to 96.1 %, and that is optimistic because overlapping windows from only 10 clips are not independent.

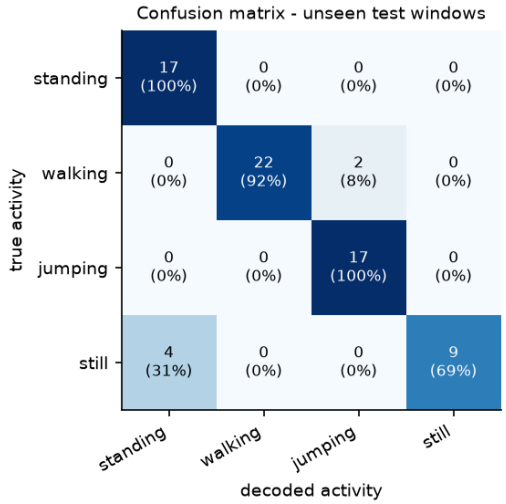


Figure 3. Confusion matrix on the unseen test windows (counts and row percentages).

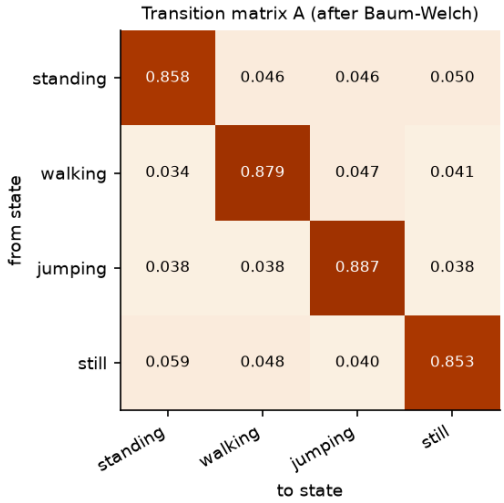


Figure 4. Transition matrix A after Baum-Welch (rows: from, columns: to).

Overall, 65 of 71 windows (91.5 %) are correct. The same model scores 98.8 % on its own training windows (it was initialized from them) and 98.4 % in leave-one-clip-out on the training clips, so 91.5 % on unseen sessions is the figure to trust. Per-window emission argmax without the transition matrix also gets 65 of 71 per clip, so Viterbi smoothing adds no window there; decoding one concatenated test sequence (shuffled clip order) gives 66 of 71 (93.0 %) with Viterbi against 65 of 71 (91.5 %) without transitions, a gain of one window. **Standing and jumping were never missed** (17 of 17 each), and the amplitude features separate jumping from the rest (Figure 5; the top four training features by Fisher ratio are all amplitude features: `acc_rms_z`, `acc_sma`, `acc_std_z`, `acc_energy_z`). **Still is the hardest** (sensitivity 0.692, 9 of 13): all 4 missed windows were decoded as standing, 2 in `still_01` and 2 in `still_12`. The 2 walking errors are windows of `walking_13` decoded as jumping. Tracing the 6 errors (`results/test_errors.csv`), **4 of 6 are in the first two windows (first 3 s) of a clip** (`still_12` and `walking_13`, windows 0 and 1), and the other 2 are the last two windows of `still_01` (windows starting at 5.0 s and 6.0 s of an 8.6 s clip). Residual phone-handling movement at the ends of clips is a plausible explanation, but I did not test it.

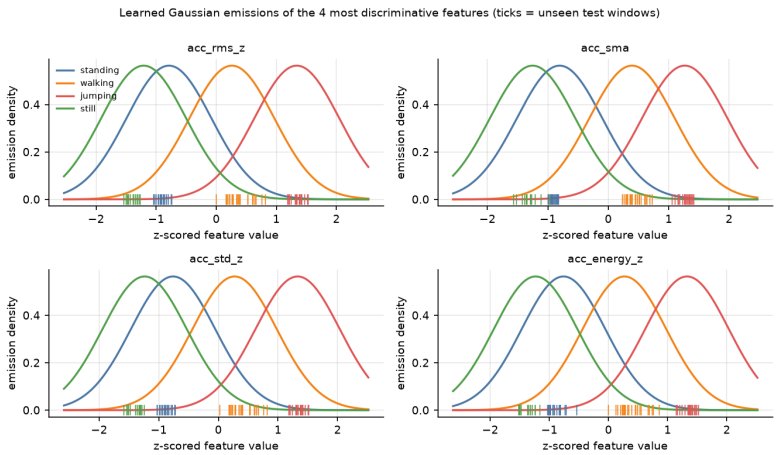


Figure 5. Learned Gaussian emission densities of the four most discriminative features (Z-scored), with the unseen test windows marked as ticks. The four states have the same width because the variance floor is binding (Section 3.3).

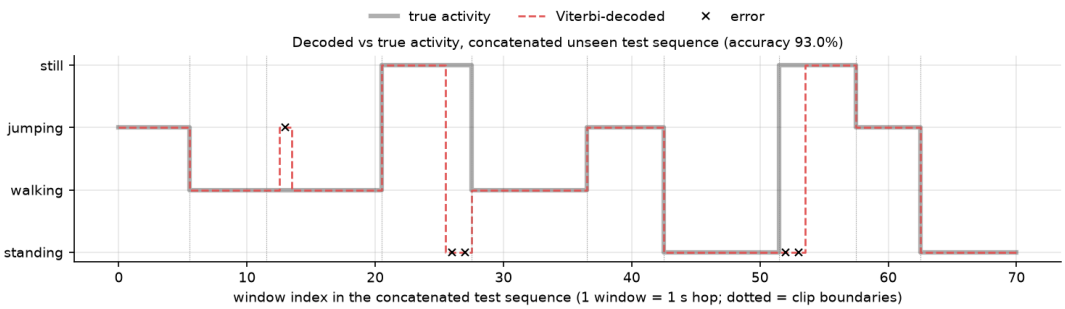


Figure 6. True versus Viterbi-decoded activity for one concatenated unseen test sequence (71 windows, shuffled clip order); crosses mark the five errors and dotted lines mark clip boundaries.

The transition matrix (Figure 4) is strongly diagonal (0.853 for still to 0.887 for jumping, an expected dwell of 6.8 to 8.8 windows) with small, near-uniform off-diagonal entries (0.034 to 0.059). Figure 6 shows the decoded sequence following the true activity. **Variance floor.** A check made after the evaluation (`results/variance_floor_sensitivity_posthoc.csv`) gives test accuracies between 91.5 % and 100 % across the five candidate floors, and the floor chosen on the training clips (0.5) is the lowest of them. I kept 0.5, because choosing a floor from test accuracy would be tuning on the test set; the result also shows that leave-one-clip-out accuracy within the training sessions did not predict the ranking of floors on unseen sessions.

5. Discussion and Conclusion

- **Easiest and hardest.** Jumping is the easiest: sensitivity 1.000 and specificity 0.963, with an accelerometer-magnitude std (median per clip 5.545) far above walking (0.778), standing (0.089) and still (0.040); the top four training features by Fisher ratio are all amplitude features. Standing was never missed either (17 of 17) but has the lowest specificity (0.926), because it absorbed the still errors. Still is the hardest (sensitivity 0.692, 9 of 13 windows) and all 4 errors were decoded as standing: the median std of still and standing differs only by a factor of 2.2. Walking reached 0.917 (22 of 24); its 2 misses are the first two windows of `walking_13`, decoded as jumping. The errors are concentrated at the ends of clips: 4 of the 6 test errors are in the first two windows of a clip and the other 2 are the last two windows of `still_01`, so phone handling at the ends of a recording is a likely but untested cause.
- **Transition probabilities.** A is strongly diagonal (0.858 for standing, 0.879 for walking, 0.887 for jumping, 0.853 for still; off-diagonal entries 0.034 to 0.059; expected dwell 6.8 to 8.8 windows of 1 s), but this describes how I built the data, not how people behave: every clip is 5 to 10 s long, no activity change happens inside a clip, and the training sequences are shuffled whole clips. The off-diagonal entries therefore come from the shuffling, and I cannot claim that still to standing is more natural than still to jumping. Learning realistic transitions needs continuous recordings with natural switches.
- **Noise and sampling rate.** The measured rate was 100.118 Hz with 9.98 to 9.99 ms between samples, so the Nyquist frequency is 50.06 Hz, far above the rhythms I measured (jumping about 2 Hz, walking median 1.5 Hz on y and z) and above the 0.5 to 10 Hz band used for the dominant-frequency features. I did not test lower rates, so I cannot say how far the rate could be reduced. The difficulty of still versus standing comes from noise and hand tremor, not from the sampling rate: the median dominant frequency of still is 5.0 to 7.0 Hz per axis, which is noise, not a rhythm. The 2 s window limits the frequency resolution to 0.5 Hz, and only 59.8 % of walking windows have a dominant bin between 1.0 and 2.5 Hz, which fits slow indoor walking.
- **Improvements and limits.** (1) Record more data: still has 88.2 s, 1.8 s below the 90 s target. (2) Record continuous sessions with natural activity changes and learn A from them, or use a hidden semi-Markov model with explicit dwell times. (3) Add features that separate still from standing, such as gravity direction from the uncalibrated accelerometer, and try full-covariance or mixture emissions. (4) Evaluate across people, phones and placements. The results have limits: one person, one phone and one place; 10 test clips and 71 overlapping windows; Baum-Welch raised the training log-likelihood by 32.18 in 4 iterations and left the test predictions unchanged; 79.5 % of the state variances sit at the variance floor; the floor picked on training data (0.5) gave the lowest test accuracy of the five candidates, and I kept it because choosing it from test results would be tuning on the test set; and training windows score 98.8 % against 91.5 % on test. Because I tried no other model family, I cannot separate limits of the data from limits of the algorithm.

In conclusion, a from-scratch HMM with 44 time and frequency features recognized four activities on unseen recordings with 91.5 % window accuracy (65 of 71). Jumping and standing were never missed and still was the weak class. The results are limited by a small single-person dataset, a model that behaves like a nearest-mean classifier, and errors concentrated at the ends of clips. The next step is more data, recorded continuously.